

“Try & Score” — Sentence-Scoring API, per Category

How the doctor page’s Try & Score button scores a single edited sentence, and how the call is routed per clinical category to the right prompt.

Endpoint POST /api/doctor/score-sentence **Model** Azure OpenAI GPT-4o **Code wins** if docs diverge

What it does

On the doctor dashboard, when a doctor edits a sentence and clicks **Try & Score**, the app sends that sentence + its category to the backend, which scores it 0–5 with GPT-4o using the **per-domain scoring prompt** and returns a score + explanation. It is an on-the-fly, single-sentence version of the batch AI scoring step.

End-to-end call chain

```
[Doctor page] PhysicianReportsModifiedV41Timothy.tsx
  button → scoreSentence(newSentence, classNumber)    classNumber = TOPIC_TO_CLASS[topicName]
    ▼ hooks/useDoctorData.tsx (scoreSentence)
  POST /api/backend/doctor/score-sentence  body = { sentence, class_ }
    ▼ app/Backend/routes_doctor.py score_sentence() [POST /api/doctor/score-sentence]
  domain = class_ or "cp" → domain_map → domain_short (cp/le/ed/inc/ius)
  prompt = load_prompt("scoring", domain_short, "1") → ai_pipeline/prompts/scoring/<domain>.py
  result = call_llm(client, model, params, prompt, text=sentence) → Azure OpenAI GPT-4o
    ▼ { score, sentence, explanation }
```

Request / Response

Field	Type	Meaning
sentence	string	the (edited) sentence to score
class_	string (optional)	the clinical category; if omitted the backend defaults to cp

Response (SentenceScoringResponse): { score: 0–5, sentence, explanation }.

Per-category routing (the core)

The category name is translated **twice**: the frontend maps the display topic → a long class name (class_), the backend maps that → a short domain, which selects the per-domain prompt file.

Display topic (UI)	class_ sent	→ short domain	scoring prompt file
Cancer Prognosis	cancer_prognosis	cp	scoring/cp.py
Life Expectancy	life_expectancy	le	scoring/le.py
Erectile Dysfunction	erectile_dysfunction_potency	ed	scoring/ed.py
Urinary Incontinence	continence	inc	scoring/inc.py
Irritative Symptoms	irritative_urinary_symptoms_...	ius	scoring/ius.py

The backend `domain_map` is tolerant — it also accepts the short names (`cp` ... `ius`) and numeric codes (`1` → `cp`, `2` → `inc`, `3` → `ed`, `4` → `ius`, `5` → `le`). The active doctor page (V41Timothy) always sends the **long names**, so the numeric branch is an unused fallback. Unknown/missing `class_` → `cp` .

Scoring parameters (what GPT-4o is called with)

- `model` = `settings.azure_openai_model` · `api_version` = `settings.azure_openai_api_version`
- `params` = `{ "max_tokens": 4096, "temperature": 0.3, "top_p": 0.4, "seed": 0 }`
- `prompt` = `load_prompt("scoring", <domain>, "1")` — prompt **version “1”** per domain.
- `call` = `call_llm(client, model, params, prompt, text=sentence)` — the same `call_llm` the batch pipeline uses (`ai_pipeline/scoring.py::run_scoring`).

Score rubric (0–5)

- 0 no mention
- 1 mention, no risk
- 2 qualitative only
- 3 numeric, no timeline
- 4 numeric + timeline
- 5 patient-specific estimate + timeline

Parameter-correctness review — verified

- ✓ `sentence` is sent as `text=` to `call_llm` — correct.
- ✓ `class_` (category) is mapped to the correct short domain for all 5 categories; the right per-domain prompt is loaded. No cross-category leakage.
- ✓ **Category is type-safe:** `topicName` is typed `TopicName` (a 5-value union), only iterated from `ALL_TOPICS` ; `TOPIC_TO_CLASS` is `Record<TopicName, string>` . So `class_` is never `undefined` — the backend’s “default to cp” branch is unreachable (defensive only).
- ✓ **Consistent with the batch pipeline:** `ai_pipeline/config.yaml` uses `max_tokens 4096`, `temperature 0.3`, `top_p 0.4`, `seed 0` and `prompts.scoring: "1"` for all five domains — identical to the on-the-fly endpoint. `model` is `gpt-4o` on both sides.

Key files

- **Frontend:** `PhysicianReportsModifiedV41Timothy.tsx` (`TOPIC_TO_CLASS` , call site), `hooks/useDoctorData.tsx` (`scoreSentence` , types).
- **Backend:** `app/Backend/routes_doctor.py` (`score_sentence` , `domain_map`).

- **AI pipeline:** `ai_pipeline/scoring.py` (`run_scoring`), `utils/prompts.py` (`load_prompt`), `prompts/scoring/<domain>.py` , `ai_pipeline/llm.py` (`call_llm`).